

# PHP

## Practicals with Answers

[Server-Side Scripting](#) | [Forms](#) | [MySQL](#) | [OOP](#) | [Sessions](#)

---

Variables, Data Types & Operators · Control Flow, Loops & Functions · Arrays & Built-in Functions · PHP Forms & MySQL CRUD · PHP Sessions, Cookies & OOP

## Practical 1: Variables, Data Types & Operators

**Aim:** Learn PHP syntax, variable declaration, data types, and operators.

### Code:

```
<?php
$name = "Yug";           // string
$age = 20;               // int
$marks = 88.5;           // float
$active = true;          // bool

echo gettype($name);     // string
var_dump($marks);        // float(88.5)

echo $name . " Patel";   // concatenation
$name .= " Patel";       // append

echo strlen($name);      // 9
echo strtoupper($name);  // YUG PATEL
echo substr($name, 0, 3); // Yug

echo 10 % 3;             // 1
echo 2 ** 8;             // 256

var_dump(10 == "10");    // bool(true) loose
var_dump(10 === "10");   // bool(false) strict

$bio = <<<EOT
Name: $name
Age: $age
EOT;
echo $bio;
?>
```

### Explanation:

PHP is server-side. Variables: **\$varName**. Types: string, int, float, bool, array, null, object. **.**: string concatenation. **.=**: append. **==**: loose comparison. **===**: strict (type+value). **var\_dump()**: shows type+value. **gettype()**: returns type name. Heredoc: **<<<EOT...EOT** for multiline strings with variable interpolation.

**Output:** gettype: 'string'. var\_dump: float(88.5). 10==='10' is false (strict). Heredoc prints multiline with variable substitution.

## Practical 2: Control Flow, Loops & Functions

**Aim:** Use if/else, switch, for/while loops and define reusable functions.

### Code:

```
<?php
$marks = 82;
if ($marks >= 90)      echo "A";
elseif ($marks >= 75)  echo "B"; // outputs B
else                   echo "F";

$status = ($marks >= 75) ? "Pass" : "Fail";
$name = $_GET['name'] ?? "Guest"; // null coalescing

switch ("Mon") {
    case "Mon": echo "Monday"; break;
    default:    echo "Other";  break;
}

for ($i=1; $i<=5; $i++) echo $i." "; // 1 2 3 4 5

foreach (["Yug","Alice"] as $i=>$n) {
    echo "$i: $n
";
}

function greet(string $name, string $msg="Hello"): string {
    return "$msg, $name!";
}
```

```

}
echo greet("Yug");           // Hello, Yug!
echo greet("Yug","Hi");      // Hi, Yug!

function sumAll(int ...$nums): int {
    return array_sum($nums);
}
echo sumAll(1,2,3,4,5); // 15

$double = fn($n) => $n * 2;
echo $double(7);           // 14
?>

```

#### Explanation:

PHP uses **\$var**. **foreach (\$arr as \$key => \$val)**: iterate arrays. **?:** ternary. **??**: null coalescing (right if left is null). **switch/case/break**: multi-branch. Type hints: **function fn(string \$a): int**. **...**: variadic args as array. **fn(\$x) => expr**: arrow function. **array\_sum()**: sum values.

**Output:** Marks=82 outputs 'B'. foreach lists 0:Yug, 1:Alice. greet uses default param. sumAll=15. arrow function double(7)=14.

### Practical 3: Arrays & Built-in Functions

**Aim: Work with indexed arrays, associative arrays, and array functions.**

#### Code:

```

<?php
$fruits = ["Mango","Apple","Banana"];
echo count($fruits); // 3
$fruits[] = "Grape"; // append

sort($fruits); // ascending
$sliced = array_slice($fruits,1,2);
$has = in_array("Apple",$fruits); // true

$student = ["name"=>"Yug", "marks"=>88, "branch"=>"IT"];
echo $student["name"]; // Yug
$student["cgpa"] = 8.5;

echo implode(" ", array_keys($student));
// name, marks, branch, cgpa

$class = [
    ["name"=>"Yug", "marks"=>88],
    ["name"=>"Alice", "marks"=>92],
    ["name"=>"Bob", "marks"=>74],
];
usort($class, fn($a,$b) => $b["marks"] - $a["marks"]);
// sorted: Alice 92, Yug 88, Bob 74

$marks_list = array_column($class,"marks");
$above80 = array_filter($marks_list, fn($m)=>$m>80);
$doubled = array_map(fn($m)=>$m*2, $marks_list);

print_r($above80); // [88, 92]
?>

```

#### Explanation:

**\$arr[] = val**: append element. **count()**: length. **sort/rsort**: sort in-place. **array\_reverse/slice/search**: manipulation. **in\_array(val,arr)**: check existence. **implode(glue,arr)**: join to string. **explode(delim,str)**: split to array. **usort(arr,fn)**: custom sort. **array\_column**: extract field from 2D array. **array\_map/filter**: transform or filter.

**Output:** Sorted: Alice:92, Yug:88, Bob:74. above80=[88,92]. doubled=[176,184,148]. array\_keys gives 'name,marks,branch,cgpa'.

### Practical 4: PHP Forms & MySQL CRUD

**Aim: Handle HTML form data and perform Create, Read, Update, Delete with MySQL.**

#### Code:

```

<?php
$conn = new mysqli("localhost","root","","college_db");
if ($conn->connect_error) die("Connection failed");

// CREATE
if ($_SERVER["REQUEST_METHOD"]=="POST") {
    $name = $conn->real_escape_string($_POST["name"]);
    $marks = (float)$_POST["marks"];
    $sql = "INSERT INTO students (name,marks) VALUES ('$name',$marks)";
    echo $conn->query($sql) ? "Added!" : $conn->error;
}

// READ
$result = $conn->query("SELECT * FROM students ORDER BY marks DESC");
echo "<table border='1'>
    <tr><th>ID</th><th>Name</th><th>Marks</th><th>Del</th></tr>";
while ($row = $result->fetch_assoc()) {
    echo "<tr>
        <td>{$row['student_id']}</td>
        <td>{$row['name']}</td>
        <td>{$row['marks']}</td>
        <td><a href='?del={$row['student_id']}'>Delete</a></td>
    </tr>";
}
echo "</table>";

// UPDATE
if (isset($_GET["upd"])) {
    $id = (int)$_GET["upd"];
    $m = (float)$_POST["marks"];
    $conn->query("UPDATE students SET marks=$m WHERE student_id=$id");
}

// DELETE
if (isset($_GET["del"])) {
    $id = (int)$_GET["del"];
    $conn->query("DELETE FROM students WHERE student_id=$id");
}
$conn->close();
?>

```

#### Explanation:

**new mysqli(host,user,pass,db):** connect. **\$conn->query(\$sql):** execute. **\$result->fetch\_assoc():** row as array. **real\_escape\_string():** basic injection prevention. **\$\_POST[key]:** form data. **\$\_GET[key]:** URL param. **\$\_SERVER[REQUEST\_METHOD]:** GET or POST. **isset():** check key exists. Cast inputs to int/float in SQL for safety.

**Output:** HTML table shows all students ordered by marks. Delete link removes row. POST inserts new student. Update modifies marks. **real\_escape\_string** prevents basic injection.

## Practical 5: PHP Sessions, Cookies & OOP

**Aim:** Use sessions for login, cookies for persistence, and OOP concepts.

#### Code:

```

<?php
session_start(); // Must be FIRST line

// Login
if (isset($_POST["login"])) {
    if ($_POST["user"]=="admin" && $_POST["pass"]=="1234") {
        $_SESSION["user"] = "admin";
        $_SESSION["logged_in"] = true;
    }
}
echo ($_SESSION["logged_in"] ?? false)
    ? "Welcome, ".$_SESSION["user"]
    : "Not logged in";
// Logout: session_unset(); session_destroy();

```

```
// Cookie
setcookie("theme","dark", time()+86400*30, "/");
echo $_COOKIE["theme"] ?? "light";    // dark

// OOP
class Student {
    public    string $name;
    protected float $marks;

    public function __construct(string $name, float $marks) {
        $this->name = $name;
        $this->marks = $marks;
    }
    public function grade(): string {
        return $this->marks >= 90 ? "A"
            : ($this->marks >= 75 ? "B" : "C");
    }
    public function __toString(): string {
        return "{$this->name} ({ $this->grade()})";
    }
}

class PlacedStudent extends Student {
    public function __construct(string $name, float $marks,
                                public string $company) {
        parent::__construct($name, $marks);
    }
    public function info(): string {
        return parent::__toString()." @ {$this->company}";
    }
}

$s = new PlacedStudent("Yug", 85.0, "TCS");
echo $s->info();    // Yug (B) @ TCS
echo $s->name;      // Yug (public access)
?>
```

#### Explanation:

**session\_start()**: must be first. **\$\_SESSION[key]**: data across pages per user. **session\_destroy()**: logout. **setcookie(name,val,expiry)**: client storage. **\$\_COOKIE[key]**: read cookie. OOP: **public/protected/private** access. **\$this->prop**: instance property. **parent::\_\_construct()**: call parent. **extends**: inheritance. **\_\_toString()**: called when object echoed.

**Output:** Session stores login across pages. Cookie persists 30 days. PlacedStudent outputs 'Yug (B) @ TCS'. grade() method inherited. name is public so directly accessible.

---

-- End of Practicals --